

UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Relatório do Trabalho Prático 2 - Gestão de Farmácia

Afonso Esteves - 55047
Felipe Dourado - 53394
Tiago Pereira - 55019

11 de dezembro de 2025

Conteúdo

Resumo **4**

1 Introdução **5**
1.1 Âmbito, enquadramento e motivação 5

2 Contextualização do Projeto **5**
2.1 Identificação dos Desenvolvedores 5
2.1.1 Desenvolvedores Envolvidos 5
2.2 Contextualização Académica 5

3 Especificações Técnicas **6**
3.1 Ambiente de Desenvolvimento 6
3.2 Tecnologias e Bibliotecas 6
3.3 Práticas de Engenharia 6

4 Utilização **7**
4.1 Requisitos Prévios 7
4.2 Compilação e Execução 7
4.3 Execução de Testes Unitários 8
4.4 Geração de Documentação JavaDoc 8

5 Estrutura da Solução Construída **9**
5.1 Visão Geral 9
5.2 Detalhamento dos Ficheiros 9

6 Testes e Documentação **11**
6.1 Testes Unitários 11
6.2 Documentação JavaDoc 11

7 Funcionalidades Implementadas **12**
7.1 Gestão de Clientes 12
7.2 Gestão de Produtos 12
7.3 Gestão de Encomendas 12
7.4 Persistência e Validações 13

8 Diagrama de Classes **14**
8.1 Ferramenta Utilizada 14
8.2 Descrição do Diagrama 14
8.3 Legenda do Diagrama 17
8.3.1 Organização em Pacotes 17

8.3.2 Modificadores de Visibilidade 17
8.3.3 Tipos de Relacionamentos 17
8.3.4 Elementos Especiais 19
8.3.5 Multiplicidades 19
8.3.6 Atributos e Métodos Principais 19

9 Considerações de Implementação **20**
9.1 Encapsulamento e Validações 20
9.2 Validação de Dados de Entrada 20
9.3 Gestão Automática de Stock 20
9.4 Persistência e Identificadores 20
9.5 Sincronização entre Componentes 21

10 Conclusão **21**

11 Referências Bibliográficas **22**

Acrónimos **22**

Resumo

Este relatório documenta a evolução de um sistema de gestão de farmácia desenvolvido no âmbito da disciplina de Programação Orientada a Objetos. A segunda entrega introduziu uma interface gráfica baseada em Java Swing, persistência de dados através de serialização de objetos, testes unitários JUnit 5 e documentação com JavaDoc. O código foi reorganizado em pacotes (`app`, `modelo`, `gui`, `persistencia`, `test`) e beneficiou de refatoração orientada por princípios de POO, resultando numa aplicação mais robusta, reutilizável e preparada para futuras extensões.

Palavras-chave e *Keywords*

Java; Swing; GUI; Programação Orientada a Objetos; Persistência de Dados; Serialização; JUnit; Javadoc.

1 Introdução

1.1 Âmbito, enquadramento e motivação

Este documento descreve o Trabalho Prático 2 (TP2) da disciplina, que dá continuidade ao sistema originalmente desenvolvido no TP1. O objetivo é consolidar competências de POO ao migrar uma aplicação de consola para uma interface gráfica, garantindo persistência de dados e qualidade através de testes e documentação formal. A motivação passa por aproximar o projeto de cenários reais: uma interface amigável, dados duráveis, testes automatizados e documentação técnica.

2 Contextualização do Projeto

2.1 Identificação dos Desenvolvedores

Os autores são estudantes do curso de Informática Web, Móvel e na Nuvem da Universidade da Beira Interior (UBI), inscritos na unidade curricular de Programação Orientada a Objetos.

2.1.1 Desenvolvedores Envolvidos

- **Afonso Esteves (55047)**
- **Felipe Dourado (53394)**
- **Tiago Pereira (55019)**

2.2 Contextualização Académica

O projeto foi desenvolvido com os seguintes objetivos principais:

- Desenvolver GUIs com Java Swing e programação orientada a eventos.
- Aplicar herança, interfaces, sobrecarga e sobreposição.
- Implementar persistência de dados com serialização de objetos.
- Criar testes unitários com JUnit 5.
- Gerar documentação JavaDoc.
- Produzir documentação escrita consolidando as decisões de design.

3 Especificações Técnicas

3.1 Ambiente de Desenvolvimento

- **Sistema Operativo**: macOS 15 (Darwin 25.0.0).
- **Shell**: `/bin/zsh`, utilizado para compilação, execução, testes e geração de documentação.
- **IDE / Editores**: IntelliJ IDEA e Cursor, com suporte a funcionalidades de reestruturação, inspeções e integração com VCS.
- **Controlo de Versões**: Git (repositório local e integração com plataformas remotas).

3.2 Tecnologias e Bibliotecas

- **Linguagem de Programação**: Java 21, permitindo explorar recursos modernos da linguagem mantendo compatibilidade retroativa.
- **Biblioteca Gráfica**: Java Swing e AWT para construção da interface (componentes como `JFrame`, `JPanel`, `JTable`, `JDialog`).
- **Persistência**: Serialização nativa com `ObjectOutputStream`/`ObjectInputStream`, garantindo persistência sem recurso a bases de dados relacionais ou NoSQL.
- **Testes**: Framework JUnit 5, distribuída através do artefacto `junit-platform-console-sta` permitindo execução em linha de comandos e integração CI.
- **Documentação**: JavaDoc para produção de documentação HTML da API.
- **Modelação UML**: PlantUML através da extensão PlantUML4idea do IntelliJ IDEA para criação e renderização de diagramas de classes.
- **Bibliotecas Standard**: `Java.util` (coleções, `Optional`), `java.io` (streams de objetos) e `java.awt.event` (listeners).

3.3 Práticas de Engenharia

- **Arquitetura em Camadas**: Separação clara entre domínio, vista e persistência.

- **POO Avançada**: Utilização de herança (`Pessoa` →`Cliente`), abstrações (`Vendavel`), sobrecarga/sobrescrita (`Encomenda`), encapsulamento e composição (`Cliente`–`Morada`).
- **Programação Orientada a Eventos**: Modelo de listeners para captar interações na GUI.
- **Testes Automatizados**: Validação da lógica de negócio com JUnit e execução regular durante o desenvolvimento.
- **Documentação Contínua**: Manutenção do JavaDoc e do relatório técnico (`CONTEXTO.md`) alinhados com o estado do código.

4 Utilização

4.1 Requisitos Prévios

- **Java Development Kit (JDK) 17 ou superior**: necessário para compilar e executar.
- **Acesso à linha de comandos**: Terminal (macOS) ou CMD/PowerShell (Windows).

4.2 Compilação e Execução

1. Navegar até ao diretório raiz do projeto (`TP-02/`).
2. Compilar código de produção:

   ```
   javac -cp "lib/junit-platform-console-standalone-1.10.2.jar:src" -d out \
     $(find src -name "*.java" ! -path "*/test/*")
   ```

   Alternativamente, sem utilizar `find`:

   ```
   javac -cp "lib/junit-platform-console-standalone-1.10.2.jar:src" -d out \
     src/app/*.java src/modelo/*.java src/gui/*.java src/persistencia/*.java
   ```

3. Compilar classes de teste:

   ```
   javac -cp "lib/junit-platform-console-standalone-1.10.2.jar:out" -d out \
     $(find src/test -name "*.java")
   ```

4. Executar a aplicação GUI:

   ```
   java -cp "out" app.Main
   ```

4.3 Execução de Testes Unitários

1. Após a compilação, executar:

```
java -jar lib/junit-platform-console-standalone-1.10.2.jar \
  --class-path out --scan-class-path
```

4.4 Geração de Documentação JavaDoc

1. Na raiz do projeto:

```
javadoc -d doc -sourcepath src -subpackages app modelo gui persistencia
```

2. O resultado fica em `doc/index.html`.

5 Estrutura da Solução Construída

5.1 Visão Geral

O código encontra-se organizado por responsabilidade:

- **Pacote** `app`: ponto de entrada `Main`.
- **Pacote** `modelo`: domínio da aplicação (entidades e regras).
- **Pacote** `gui`: interface Swing (janela principal e painéis).
- **Pacote** `persistencia`: gestão de ficheiros para serialização.
- **Pacote** `test`: testes JUnit 5.

5.2 Detalhamento dos Ficheiros

- `src/app/Main.java`: Carrega dados, inicializa `Farmacia` e arranca a GUI. Lança a aplicação na Event Dispatch Thread.
- `src/modelo/Pessoa.java`: Classe abstrata base com nome e NIF. Obriga subclasses a implementar `toString()`.
- `src/modelo/Cliente.java`: Extende `Pessoa`, adiciona `Morada` e serialização. Representa clientes da farmácia.
- `src/modelo/Morada.java`: Value object com rua, cidade e código postal.
- `src/modelo/Vendavel.java`: Interface com `getNome()` e `getPreco()`.
- `src/modelo/Produto.java`: Implementa `Vendavel`; gere ID, preço e stock, com métodos de validação.
- `src/modelo/Encomenda.java`: Relaciona cliente e produtos, controla estados, calcula preço total, oferece construtores sobrecarregados.
- `src/modelo/Farmacia.java`: Núcleo da lógica; gere listas, valida operações, ajusta stock e estados.
- `src/persistencia/GestorDados.java`: Serializa/desserializa dados para `farmacia.dat`; atualiza contadores de IDs.
- `src/gui/FarmaciaGUI.java`: `JFrame` com `JTabbedPane`; trata encerramento com opção de guardar.

- `src/gui/PainelGestaoClientes.java`: Tabela e formulários para CRUD de clientes, com validação de NIF (9 dígitos) e código postal (0000-000), permitindo correção de NIF apenas neste contexto.
- `src/gui/PainelGestaoProdutos.java`: Gestão de produtos com validação de stock/preço e atualização da tabela.
- `src/gui/PainelGestaoEncomendas.java`: Gestão completa de encomendas (seleção de cliente, seleção/pesquisa de produtos, certificação de stock, atualização de estado e sincronização com painel de produtos), apresentando o cliente apenas para consulta durante a edição e com diálogo de gestão que oferece ações de adicionar, remover e guardar alterações.
- `src/test/FarmaciaTest.java`: Testes unitários que asseguram regras de negócio (clientes, encomendas, stock).
- `lib/junit-platform-console-standalone-1.10.2.jar`: Dependência JUnit 5.
- `doc/`: JavaDoc gerado.

6 Testes e Documentação

6.1 Testes Unitários

Os testes unitários foram implementados utilizando JUnit 5 e encontram-se no ficheiro `src/test/FarmaciaTest.java`. Estes testes cobrem operações críticas do sistema, incluindo:

- Adição e remoção de clientes, garantindo que não são permitidos clientes duplicados (mesmo NIF).
- Registo de encomendas e verificação da dedução automática de stock dos produtos envolvidos.
- Remoção de encomendas e validação da reposição correta do stock.
- Atualização de produtos de uma encomenda, incluindo casos de stock insuficiente.

A execução dos testes é realizada através da linha de comandos utilizando o artefacto JUnit standalone, permitindo validação contínua da lógica de negócio durante o desenvolvimento.

6.2 Documentação JavaDoc

A documentação da API foi gerada automaticamente utilizando JavaDoc para todos os pacotes do projeto (`app`, `modelo`, `gui`, `persistencia`). Esta documentação facilita a compreensão da estrutura do código, dos métodos disponíveis e dos relacionamentos entre classes, sendo essencial para manutenção e evolução futura do sistema.

7 Funcionalidades Implementadas

O sistema desenvolvido oferece um conjunto abrangente de funcionalidades organizadas em três áreas principais:

7.1 Gestão de Clientes

- Interface gráfica com tabela para visualização de todos os clientes registados.
- Operações CRUD completas: criação, leitura, atualização e remoção de clientes.
- Validação rigorosa de dados: NIF com exatamente 9 dígitos e código postal no formato 0000-000.
- Garantia de unicidade do NIF, impedindo registos duplicados.
- Possibilidade de correção de NIF durante a edição no painel de gestão de clientes.

7.2 Gestão de Produtos

- Interface tabular para visualização do catálogo de produtos.
- CRUD completo com validações de integridade: preço e stock não podem ser negativos.
- Gestão de stock com atualização dinâmica da interface após alterações.
- Sincronização automática entre painéis quando o stock é alterado através de encomendas.

7.3 Gestão de Encomendas

- Registo de novas encomendas com seleção de cliente (pesquisa por NIF) e produtos (pesquisa por nome ou ID).
- Validação em tempo real do stock disponível antes de confirmar a encomenda.
- Edição completa de encomendas existentes, incluindo:
  - Atualização do estado da encomenda (Em processamento, Pago, Enviado, Entregue).

  - Diálogo dedicado para gestão de produtos da encomenda (adicionar, remover, ajustar quantidades).
  - Apresentação do cliente apenas para consulta durante a edição.

- Remoção de encomendas com reposição automática do stock dos produtos envolvidos.

- Sincronização imediata entre os painéis de encomendas e produtos para refletir alterações de stock.

7.4 Persistência e Validações

- Persistência automática dos dados através de serialização de objetos em `farmacia.dat`.

- Diálogo de confirmação ao encerrar a aplicação, permitindo escolher se deseja guardar os dados.

- Carregamento automático dos dados persistidos ao iniciar a aplicação.

- Validações abrangentes: NIF único, formato de código postal, stock suficiente, valores não negativos.

8 Diagrama de Classes

8.1 Ferramenta Utilizada

O diagrama de classes foi criado utilizando a extensão **PlantUML4idea** do IntelliJ IDEA, que integra o framework PlantUML diretamente no ambiente de desenvolvimento. Esta ferramenta permite criar diagramas UML através de uma linguagem de marcação textual, facilitando a manutenção e versionamento do diagrama através do ficheiro fonte `diagrama.puml`.

A extensão PlantUML4idea oferece funcionalidades de renderização em tempo real e exportação para diversos formatos de imagem, incluindo PNG, SVG e PDF. Para este projeto, o diagrama foi renderizado e exportado como imagem PNG (`diagrama.png`) a partir do ficheiro fonte PlantUML, garantindo alta qualidade e compatibilidade com o documento LaTeX.

8.2 Descrição do Diagrama

A Figura 1 apresenta o diagrama de classes UML que ilustra a arquitetura completa do sistema de gestão de farmácia, demonstrando a organização em pacotes (`app`, `gui`, `persistencia` e `modelo`), os relacionamentos entre classes, heranças, implementações de interfaces e composições.

O diagrama evidencia a separação de responsabilidades: o pacote `modelo` contém as entidades de domínio (`Pessoa`, `Cliente`, `Produto`, `Encomenda`, `Farmacia`), o pacote `gui` agrupa os componentes da interface gráfica (`FarmaciaGUI` e os painéis de gestão), o pacote `persistencia` centraliza a gestão de dados através de `GestorDados`, e o pacote `app` contém o ponto de entrada `Main`.

As classes principais do sistema incluem:

- `Farmacia`: Classe central que gere todas as operações de negócio, mantendo listas de clientes, produtos e encomendas, e coordenando as validações e atualizações de stock.
- `Cliente`: Extende a classe abstrata `Pessoa` e representa os clientes da farmácia, contendo informações de identificação e morada.
- `Produto`: Implementa a interface `Vendavel` e gere o catálogo de produtos com controlo de stock e preços.
- `Encomenda`: Relaciona clientes com produtos, gerindo estados de processamento e calculando preços totais.
- `GestorDados`: Classe estática responsável pela persistência através de serialização de objetos.

- `FarmaciaGUI`: Interface gráfica principal que coordena os diferentes painéis de gestão através de um `JTabbedPane`.

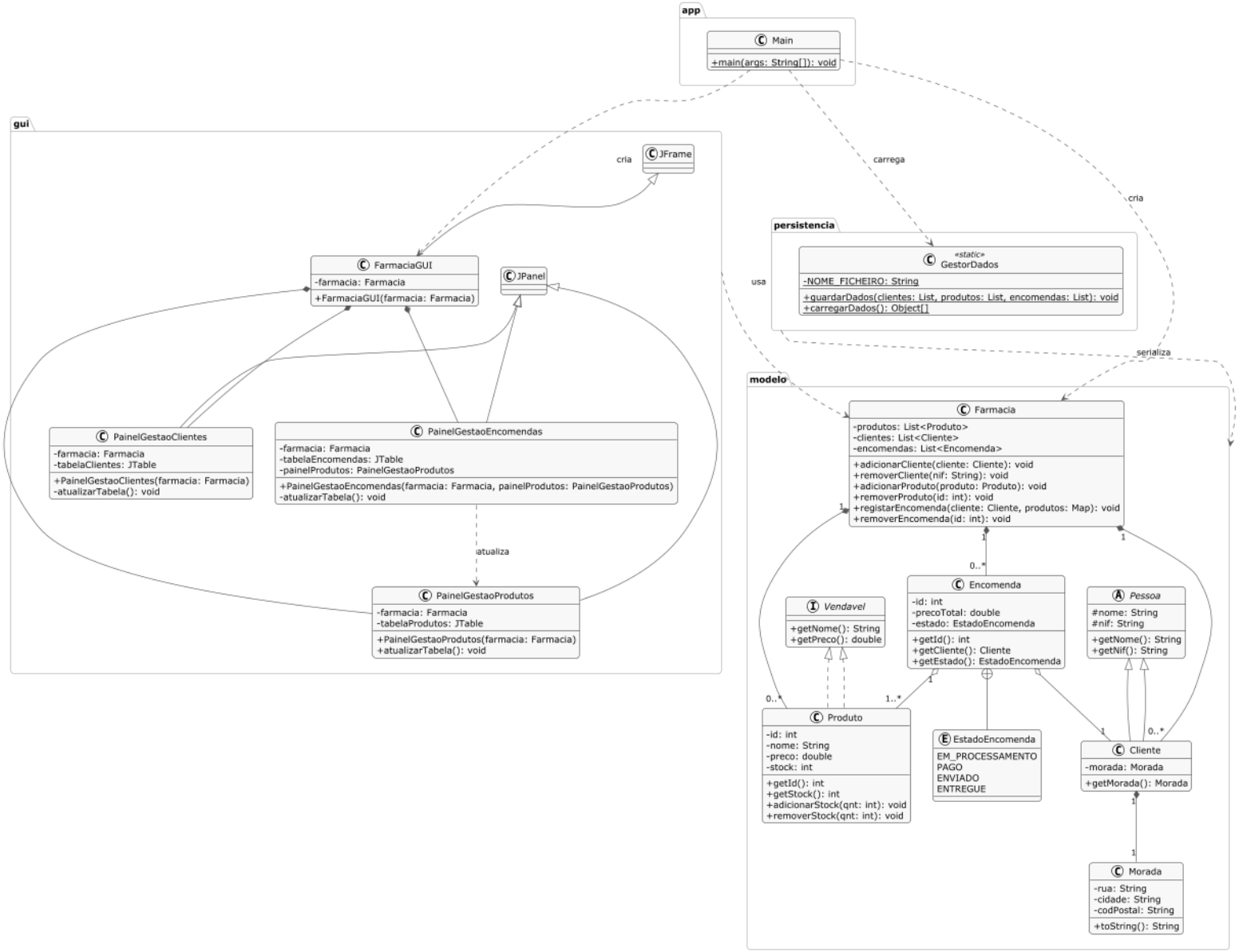


Figura 1: Diagrama de Classes da Aplicação

8.3 Legenda do Diagrama

Para facilitar a compreensão do diagrama de classes apresentado, esta secção descreve os elementos e notações UML utilizadas:

8.3.1 Organização em Pacotes

O diagrama organiza as classes em quatro pacotes principais:

- `app`: Contém a classe `Main`, ponto de entrada da aplicação responsável por inicializar o sistema e carregar dados persistidos.
- `modelo`: Agrupa as entidades de domínio que representam a lógica de negócio: `Farmacia`, `Cliente`, `Produto`, `Encomenda`, `Pessoa`, `Morada`, `Vendavel` e `EstadoEncomenda`.
- `gui`: Contém os componentes da interface gráfica: `FarmaciaGUI` e os painéis de gestão (`PainelGestaoClientes`, `PainelGestaoProdutos`, `PainelGestaoEncomendas`).
- `persistencia`: Inclui a classe `GestorDados`, responsável pela serialização e desserialização dos dados.

8.3.2 Modificadores de Visibilidade

Os atributos e métodos são representados com os seguintes símbolos:

- `+`: Visibilidade pública (público) — acessível de qualquer classe.
- `-`: Visibilidade privada (privado) — acessível apenas dentro da própria classe.
- `#`: Visibilidade protegida (protegido) — acessível na classe e nas suas subclasses.

8.3.3 Tipos de Relacionamentos

Herança (Generalização): Representada por uma seta contínua com triângulo vazio apontando para a classe base.

- `Cliente` herda de `Pessoa` — permite reutilização de código e modelação da relação "é um tipo de".
- `FarmaciaGUI` herda de `JPanel` — integração com a biblioteca Swing.

Implementação de Interface: Representada por uma seta tracejada com triângulo vazio apontando para a interface.

- `Produto` implementa `Vendavel` — garante que produtos possuem métodos `getNome()` e `getPreco()`, permitindo polimorfismo.

Composição: Representada por um losango preenchido (diamante sólido) na classe que contém.

- `Cliente` compõe `Morada` — a morada não existe independentemente do cliente (relação forte, ciclo de vida dependente).
- `Encomenda` compõe `Produto` — os produtos na encomenda são parte integrante da mesma.

Agregação: Representada por um losango vazio na classe que contém.

- `Farmacia` agrega `Cliente`, `Produto` e `Encomenda` — estas entidades podem existir independentemente da farmácia (relação fraca).

Associação: Representada por uma linha simples entre classes.

- `Encomenda` associa-se a `Cliente` — cada encomenda pertence a um cliente.
- `Encomenda` associa-se a `EstadoEncomenda` — cada encomenda possui um estado.

Dependência: Representada por uma seta tracejada com etiqueta descritiva.

- `Main` depende de `GestorDados` (etiqueta "carrega") — utiliza métodos estáticos para carregar dados.
- `Main` depende de `JFrame` (etiqueta "cria") — cria a janela principal.
- `FarmaciaGUI` depende dos painéis de gestão (etiqueta "cria") — instancia os componentes da interface.
- `PainelGestaoEncomendas` depende de `PainelGestaoProdutos` (etiqueta "atualiza") — sincroniza atualizações de stock.
- `GestorDados` depende de `Farmacia` (etiquetas "usa"e "serializa") — manipula os dados da farmácia para persistência.

8.3.4 Elementos Especiais

Classe Abstrata: `Pessoa` é marcada como abstrata (indicada por `(A)`), não podendo ser instanciada diretamente. Serve como base para `Cliente` e potenciais futuras subclasses.

Interface: `Vendavel` define um contrato que `Produto` deve implementar, permitindo polimorfismo e extensibilidade para futuros tipos de itens vendáveis.

Enumeração: `EstadoEncomenda` define os estados possíveis de uma encomenda: `EM_PROCESSAMENTO`, `PAGO`, `ENVIADO` e `ENTREGUE`, garantindo type-safety e valores fixos.

Classe Estática: `GestorDados` é marcada como `<<static>>`, indicando que todos os seus métodos são estáticos e não requerem instanciação.

8.3.5 Multiplicidades

As relações incluem multiplicidades que indicam quantidades:

- `0..*`: Zero ou mais (muitos) — `Farmacia` pode ter zero ou mais clientes, produtos e encomendas.
- `1`: Exatamente um — `Encomenda` tem exatamente um cliente e um estado.
- `1..*`: Um ou mais — `Encomenda` contém pelo menos um produto.

8.3.6 Atributos e Métodos Principais

As classes principais expõem os seguintes elementos:

- `Farmacia`: Mantém listas de clientes, produtos e encomendas; oferece métodos para adicionar/remover entidades e registar encomendas.
- `Encomenda`: Contém identificador único, preço total calculado e estado atual; relaciona-se com cliente e produtos através de um `Map`.
- `Produto`: Gere stock, preço e identificador; implementa métodos para ajustar stock (`adicionarStock`, `removerStock`).
- `Cliente`: Extende `Pessoa` com nome e NIF; contém uma `Morada` completa.
- Painéis GUI: Cada painel mantém referência à `Farmacia` e a uma `JTable` para visualização tabular dos dados.

Esta estrutura demonstra a aplicação de princípios fundamentais de POO: encapsulamento, herança, polimorfismo, composição e agregação, resultando numa arquitetura modular e extensível.

9 Considerações de Implementação

Durante o desenvolvimento do projeto, foram aplicados diversos princípios e técnicas de Programação Orientada a Objetos, resultando em decisões de implementação que merecem destaque:

9.1 Encapsulamento e Validações

O sistema utiliza extensivamente encapsulamento para proteger o estado interno das classes. Todos os atributos são privados e o acesso é controlado através de métodos públicos. Validações rigorosas garantem a consistência dos dados em todas as operações, como a verificação de que o stock nunca assume valores negativos e que os preços são sempre positivos.

9.2 Validação de Dados de Entrada

Foram implementadas regras de formato específicas para dados de clientes: o NIF deve conter exatamente 9 dígitos e o código postal deve seguir o padrão 0000-000. A correção do NIF é permitida apenas no contexto do painel de gestão de clientes, mantendo a integridade dos dados em outras partes do sistema.

9.3 Gestão Automática de Stock

O sistema implementa uma gestão inteligente do stock, deduzindo automaticamente as quantidades dos produtos quando uma encomenda é registada e repondo o stock quando uma encomenda é removida. Esta funcionalidade garante que o inventário reflete sempre o estado real dos produtos disponíveis.

9.4 Persistência e Identificadores

Após o carregamento dos dados persistidos, o sistema atualiza automaticamente os contadores estáticos de identificadores (`Produto` e `Encomenda`) para evitar colisões de IDs. Esta abordagem garante que novos registos sempre recebem identificadores únicos, mesmo após múltiplas sessões da aplicação.

9.5 Sincronização entre Componentes

Foi implementada uma sincronização eficiente entre os painéis de encomendas e produtos, garantindo que alterações de stock sejam refletidas imediatamente em todas as interfaces relevantes. Esta coordenação melhora significativamente a experiência do utilizador e previne inconsistências visuais.

10 Conclusão

O Trabalho Prático 2 alcançou com sucesso todos os objetivos propostos, resultando numa aplicação robusta e bem estruturada. A migração de uma interface de consola para uma interface gráfica baseada em Java Swing demonstrou a aplicação prática dos conceitos de programação orientada a eventos e gestão de componentes gráficos.

A implementação de persistência de dados através de serialização de objetos garantiu que os dados sejam mantidos entre sessões, enquanto os testes unitários com JUnit 5 asseguram a qualidade e consistência da lógica de negócio. A documentação JavaDoc completa facilita a manutenção e compreensão do código.

A organização do código em pacotes bem definidos (`app`, `modelo`, `gui`, `persistencia`, `test`) reflete uma arquitetura limpa e escalável, preparada para futuras extensões. A aplicação de princípios avançados de POO, como herança, interfaces, sobrecarga, sobreposição e composição, demonstra um domínio sólido dos conceitos fundamentais da disciplina.

O conhecimento consolidado neste trabalho forma uma base sólida para desafios mais complexos em Programação Orientada a Objetos e desenvolvimento de software em geral. O projeto demonstra a capacidade de transformar requisitos funcionais numa solução técnica completa, bem documentada e testada.

11 Referências Bibliográficas

- Schildt, Herbert. *Java: A Beginner's Guide*. 6th ed., McGraw-Hill Education, 2014.
- Olsson, Mikael. *Java Quick Syntax Reference*. Apress, 2014.
- Roberts, Eric S. *The Art and Science of Java*. Addison-Wesley, 2008.
- Oracle Corporation. *The Java Tutorials: Creating a GUI With Swing*. Disponível em: `https://docs.oracle.com/javase/tutorial/uiswing/`
- JUnit Team. *JUnit 5 User Guide*. Disponível em: `https://junit.org/junit5/docs/current/user-guide/`
- Oracle Corporation. *Java Platform, Standard Edition Documentation*. Disponível em: `https://docs.oracle.com/en/java/`

Acrónimos

UBI Universidade da Beira Interior

GUI Graphical User Interface

POO Programação Orientada a Objetos

JDK Java Development Kit

API Application Programming Interface

UML Unified Modeling Language

CRUD Create, Read, Update, Delete